Consegna: monorepo, CI/CD, deploy, monitoraggio

Corso di Frontend Development – Modulo 07

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. Consegna: monorepo, CI/CD, deploy, monitoraggio
1.1 Da due cartelle a un monorepo
1.2 Integrazione continua
1.3 Distribuzione
1.4 Osservare l'app in produzione

Sommario del modulo

1. Consegna: monorepo, CI/CD, deploy, monitoraggio
1.1 Da due cartelle a un monorepo
1.2 Integrazione continua
1.3 Distribuzione
1.4 Osservare l'app in produzione

Dove siamo: due progetti nello stesso repository

```
minicommerce/
  backend/
    package.json          # minicommerce-backend, typescript ~7.0.2
    package-lock.json
    src/db/schema.ts      # users, categories, products, cart_items, orders
  frontend/
    package.json          # frontend, typescript ~6.0.2
    package-lock.json
    src/lib/api.ts        # Product, Category, AuthUser, Cart, CartItem, MockOrder
```

- Due `npm install`, due lockfile, nessun comando che controlli tutto il progetto.
- Il tipo `Product` esiste **due volte**; `Cart` e `MockOrder` esistono solo nel frontend, perché il backend costruisce quelle risposte a mano. Nessuno garantisce che coincidano.
- Perfino TypeScript è a due versioni diverse: nessuno se ne accorge finché non rompe.
- E stanno arrivando desktop (modulo 2), mobile (modulo 3) e il sito vetrina (modulo 4).

Un workspace npm

```
// package.json -- nuovo, nella root del repository
{ "name": "minicommerce", "private": true,
  "workspaces": ["backend", "frontend", "desktop", "packages/*"] }
```

```
git rm backend/package-lock.json frontend/package-lock.json
npm install                  # una sola node_modules e un solo lockfile, nella root
npm run dev -w frontend      # lanciare uno script di un singolo progetto
npm run build --workspaces   # lo stesso script su tutti i progetti
```

La conversione è un vostro lavoro: oggi il repository ha ancora due `npm install` separati. Le dipendenze comuni (TypeScript, oxlint, Vitest) si dichiarano **una volta** nella root, così la versione è una sola.

packages/shared: i tipi una volta sola

```ts
// packages/shared/src/index.ts -- il contratto HTTP, non le righe del database
export interface Product {
  id: number; name: string; description: string; brand: string;
  categoryId: number; category: string;      // il nome, dalla join
  price: number; stock: number; imageUrl: string | null; createdAt: string;
}
export interface Category { id: number; name: string; description: string;
  productCount: number; createdAt: string }  // productCount lo calcola il backend
export interface AuthUser { id: number; email: string; name: string; isAdmin: boolean }
export interface CartItem { id: number; quantity: number; product: Product; subtotal: number }
export interface Cart { items: CartItem[]; total: number }
export interface MockOrder { id: number; status: 'accepted'; total: number;
  createdAt: string; message: string; items: Array<{ productId: number;
  productName: string; unitPrice: number; quantity: number }>;
  payment: { method: 'mock-card'; cardholder: string; lastFour: string } }
```

```jsonc
{ "name": "@minicommerce/shared", "private": true, "exports": { ".": "./src/index.ts" } }
// in backend e frontend: "@minicommerce/shared": "*"  -- npm lo risolve al workspace locale
```

Chi usa il pacchetto condiviso

```
// frontend/src/lib/api.ts -- le interfacce spariscono da qui
import type { Product, Category, AuthUser, Cart, MockOrder } from '@minicommerce/shared';
export type { Product, Category, AuthUser, Cart, MockOrder };

// backend/src/routes/cart.ts -- oggi { items, total } e' un oggetto costruito a mano
import type { Cart } from '@minicommerce/shared';
const carrello: Cart = { items, total };   // da qui in poi la forma la controlla tsc
res.json(carrello);
```

- Aggiungete un campo a `schema.ts` e lo dimenticate in `shared`? `tsc` fallisce in CI, non in produzione.
- In `shared` va solo ciò che viaggia su HTTP: **niente** Drizzle, niente React, niente Express. Altrimenti il frontend si tira dietro il backend.
- `passwordHash` non entra in `AuthUser`, e in `MockOrder` ci sono solo intestatario e ultime quattro cifre: il tipo condiviso è anche una difesa.

Il monorepo completo: un repository, cinque app

```
minicommerce/
  package.json            # "workspaces": [...]
  turbo.json
  openapi.yaml            # il contratto dell'API (da scrivere: e' un esercizio)
  packages/
    shared/               # @minicommerce/shared: i tipi del contratto
    config/               # tsconfig e configurazione oxlint condivisi
  backend/                # Express 5 + SQLite (il processo che scrive i dati)
  frontend/               # Vite SPA (modulo 1)        -> dist/ statico
    src-tauri/            # Tauri (modulo 2)           -> impacchetta la stessa dist/
    android/ ios/         # Capacitor (modulo 3)       -> impacchetta la stessa dist/
  desktop/                # Electron (modulo 2): guscio accanto al frontend
  web/                    # Next, vetrina (modulo 4)   -> pagine pregenerate
  blog/                   # Astro, blog  (modulo 4)    -> HTML statico
  singleton/              # Next tutto-in-uno (mod. 4) -> alternativa a backend+frontend
```

`singleton/` non è una sesta app: è l'*altra strada*, nel repository per confronto. Chi la sceglie cancella `backend/` e `frontend/`, e con loro il contratto HTTP su cui vivono desktop, mobile e native. Desktop e mobile non sono copie del frontend: ne riusano la `dist/`. L'app nativa del modulo 5 condivide il **contratto HTTP**, non il codice: può stare in un altro repository e generare il suo client da `openapi.yaml`. Una correzione in `shared` o in `lib/piattaforma.ts` arriva a web, desktop e mobile con un solo commit.

Turborepo: un comando, le dipendenze giuste

```
// turbo.json: cosa dipende da cosa, cosa si puo' cachare
{ "tasks": {
    "build":     { "dependsOn": ["^build"], "outputs": ["dist/**"] },
    "typecheck": { "dependsOn": ["^build"] },
    "test":      { "dependsOn": ["^build"] },
    "lint":      {},
    "dev":       { "cache": false, "persistent": true } } }
```

```
// package.json della root
{ "name": "minicommerce", "private": true,
  "scripts": { "dev": "turbo run dev", "lint": "turbo run lint",
    "typecheck": "turbo run typecheck", "build": "turbo run build",
    "test": "turbo run test" } }
```

`turbo run build --filter=frontend...` costruisce `frontend` e ciò da cui dipende. La cache (locale e remota) salta ciò che non è cambiato: toccate solo `blog/` e il resto non si ricompila.

Gli script che la CI deve poter chiamare

```
// frontend/package.json -- dev, build e lint ci sono: typecheck e test no
{ "scripts": { "dev": "vite", "build": "tsc -b && vite build",
    "typecheck": "tsc -b", "lint": "oxlint",
    "test": "vitest run", "test:e2e": "playwright test" } }

// backend/package.json -- qui lint e typecheck MANCANO: vanno aggiunti
{ "scripts": { "dev": "node --watch src/server.ts", "start": "node src/server.ts",
    "typecheck": "tsc --noEmit", "lint": "oxlint", "test": "vitest run",
    "db:push": "drizzle-kit push --force", "db:seed": "node src/seed.ts" } }
```

- Il backend **non ha un build**: Node 24 esegue TypeScript direttamente. Ma senza `tsc --noEmit` nessuno controlla i suoi tipi: oggi un errore si scopre solo quando la rotta va in errore.
- Regola: ogni progetto espone gli stessi nomi di script. Turborepo e la CI non devono conoscere i casi particolari.

GitHub Actions: la pipeline di base

```yaml
# .github/workflows/ci.yml
name: CI
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: npm }
      - run: npm ci
      - run: npx turbo run lint typecheck test build
      - run: npm audit --audit-level=high
      - uses: actions/upload-artifact@v4
        with: { name: frontend-dist, path: frontend/dist }
```

Ogni PR: oxlint, tipi, i test del modulo 6, build e audit delle dipendenze. Se è rosso, non si fa merge. È la rete di sicurezza che rende sostenibile il lavoro con gli agenti (modulo 8).

E2E in CI: serve anche il backend

```yaml
e2e:
  needs: check
  runs-on: ubuntu-latest
  env: { JWT_SECRET: test-secret, DATABASE_URL: /tmp/e2e.db }
  # perche' funzioni, drizzle.config.ts deve leggere DATABASE_URL:
  # oggi ha il percorso scritto a mano ('./minicommerce.db')
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: 24, cache: npm }
    - run: npm ci
    - run: npm run db:push -w backend && npm run db:seed -w backend
    - run: npm run start -w backend &
    - run: npx playwright install --with-deps chromium
    - run: npm run test:e2e -w frontend
    - uses: actions/upload-artifact@v4
      if: failure()
      with: { name: playwright-report, path: frontend/playwright-report }
```

Il database di test è un file usa e getta in `/tmp`: `db:push` e `db:seed` lo ricreano a ogni esecuzione, quindi i test partono sempre dagli stessi dati demo.

Anteprime, segreti, protezione del branch

- **Preview deployment**: Netlify, Vercel e Cloudflare Pages pubblicano il `dist/` di ogni PR a un URL temporaneo, puntato all'API di staging. Il revisore clicca invece di immaginare.
- Segreti del repository (*Settings* → *Secrets*): `JWT_SECRET` di produzione, token di deploy, chiavi di firma, e le chiavi del circuito di pagamento il giorno in cui `POST /api/orders` smetterà di essere finto. Mai nel codice, mai nei log, mai in un file `.env` committato.
- `npm ci` (non `npm install`): la CI installa **esattamente** il lockfile e fallisce se `package.json` e lockfile non concordano. Nessuna sorpresa di versione fra la vostra macchina e il server.
- **Branch protection** su `main`: CI verde obbligatoria, una revisione, niente push diretti.

Il frontend: file statici

```
npm run build -w frontend       # -> frontend/dist: index.html, assets/*.js, *.css

# nella CI, su push in main
- uses: cloudflare/wrangler-action@v3
  with:
    apiToken: ${{ secrets.CF_TOKEN }}
    command: pages deploy frontend/dist --project-name minicommerce
```

- `VITE_API_URL` si decide **al build**: una build per ambiente, oppure un `config.json` letto a runtime se volete un solo artefatto.
- **Fondamentale con BrowserRouter**: ogni percorso sconosciuto deve servire `index.html`, altrimenti `/cart`, `/about` o `/article/7` ricaricati a mano danno 404. Su Pages e Netlify è una riga di configurazione; con nginx è `try_files $uri /index.html`.
- Il blog Astro del modulo 4 segue la stessa strada: `blog/dist` è una cartella di file, si pubblica così. La vetrina Next invece è un **processo Node** da ospitare come il backend, perché le serve un server per rigenerare le pagine su richiesta: con `output: 'export'` tornerebbe statica, ma perderebbe `revalidateTag`.

Il backend: un processo e un file accanto

```dockerfile
# backend/Dockerfile -- build context: la root del monorepo
FROM node:24-slim                # non alpine: better-sqlite3 e' un modulo nativo
WORKDIR /app
COPY package.json package-lock.json ./
COPY packages ./packages
COPY backend ./backend
RUN npm ci --omit=dev -w backend --include-workspace-root
ENV NODE_ENV=production DATABASE_URL=/data/minicommerce.db
EXPOSE 3001
CMD ["node", "backend/src/server.ts"]
```

- Niente passo di compilazione: Node 24 esegue i `.ts` come sono.
- `/data` deve essere un **volume persistente**. Senza volume, il container riparte vuoto: catalogo, utenti, carrelli e ordini svaniti a ogni deploy.
- Al primo avvio su un volume nuovo servono `db:push` e `db:seed`: un passo del deploy, non magia. Fate leggere `DATABASE_URL` anche a `drizzle.config.ts`, o `drizzle-kit` crea il database fuori dal volume.

SQLite in produzione: cosa comporta

- **Un solo processo scrive**. Niente due repliche, niente autoscaling: condividere il file via rete corrompe il database. Si scala *verticalmente*, non in orizzontale.
- Quindi **niente rolling deploy**: per un istante vecchio e nuovo container toccherebbero lo stesso file. Si ferma e si riavvia, con qualche secondo di interruzione.
- **Niente serverless** (funzioni Vercel/Netlify, Workers): filesystem effimero, molte istanze in parallelo, nessun volume. SQLite lì non regge senza cambiare archivio.
- **Backup**: il file va copiato *a caldo* con `sqlite3 db .backup` o `VACUUM INTO`. Un `cp` mentre l'app scrive, con il WAL attivo, produce una copia inconsistente.
- Le immagini salvate come data URL base64 finiscono **dentro** il file: fino a 5 MB per articolo, dentro ogni backup. Spostarle su disco con un upload multipart resta un esercizio; quei file vorranno lo stesso volume.

Quando passare a Postgres

```
# drizzle.config.ts: dialect 'sqlite' -> 'postgresql', url -> DATABASE_URL
# db/index.ts: better-sqlite3 -> postgres-js (o node-postgres) + pool
# db/schema.ts: sqliteTable -> pgTable, real -> doublePrecision,
#               integer({mode:'boolean'}) -> boolean, text created_at -> timestamp
npm exec -w backend -- drizzle-kit generate   # SQL di migrazione versionato
npm exec -w backend -- drizzle-kit migrate    # applicato dal deploy
```

- Drizzle rende il passaggio un **cambio di driver più una migrazione**: le query restano quelle, i tipi delle colonne cambiano.
- Conviene quando serve più di una replica, quando il backup a caldo diventa un problema, o quando i dati non stanno più comodi su un disco solo.
- In produzione si abbandona `db:push --force` (riscrive lo schema e può perdere dati) per le migrazioni generate e committate.
- Via di mezzo: SQLite gestito (Turso/libSQL), stesso dialetto, replica inclusa.

Variabili d'ambiente: le cinque che contano

Variabile	Dove	Perché
`JWT_SECRET`	backend	oggi il default è `'dev-secret-change-me'`: con quello in produzione **chiunque** si firma un token con `isAdmin: true`
`DATABASE_URL`	backend	percorso del file sul volume persistente
`PORT`	backend	la piattaforma decide la porta, il codice la legge
`NODE_ENV`	backend	fuori da `production` il gestore errori rispedisce `detail` con il messaggio interno: informazione regalata a un attaccante
`VITE_API_URL`	frontend	una per ambiente, letta *al build*

- All'avvio, il backend dovrebbe **rifiutarsi di partire** se `NODE_ENV=production` e `JWT_SECRET` è ancora il default. Dieci righe che evitano un disastro.

Desktop: build su tre sistemi e release

```yaml
# .github/workflows/release-desktop.yml (Tauri)
on: { push: { tags: ["v*"] } }
jobs:
  build:
    strategy:
      matrix: { os: [ubuntu-22.04, windows-latest, macos-latest] }
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: npm }
      - run: npm ci
      - uses: tauri-apps/tauri-action@v0
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          TAURI_SIGNING_PRIVATE_KEY: ${{ secrets.TAURI_KEY }}
          VITE_API_URL: https://api.minicommerce.example/api
        with: { projectPath: frontend, tagName: ${{ github.ref_name }}, releaseDraft: true }
```

`projectPath` è `frontend`: lì vive `src-tauri/`, creato da `scripts/setup-tauri.mjs` (branch `desktop/tauri`). Un tag `v1.2.0` produce una release con MSI, DMG, AppImage e il manifest per l'auto-update; con Electron il passo finale è `npm run package` in `desktop/`. Senza il proxy di Vite, `VITE_API_URL` assoluto è obbligatorio.

Mobile: automatizzare gli store

- **Fastlane**: lo standard per firmare e caricare su Play Console e App Store Connect da riga di comando.
- **EAS** (Expo) e **Ionic Appflow**: build in cloud, anche iOS senza Mac. Costo mensile, tempo risparmiato.
- GitHub Actions con runner `macos-latest`: Xcode incluso, ma i minuti macOS costano 10 volte quelli Linux.
- Keystore Android e certificati Apple: nei *secrets*, codificati in base64, mai nel repo.
- Flusso tipico: tag $\rightarrow$ build $\rightarrow$ traccia interna / TestFlight $\rightarrow$ promozione manuale in produzione.
- La revisione dello store dura giorni: le correzioni urgenti vanno nel frontend web, non nell'app.

Ambienti e feature flag

Ambiente	Scopo	API	Chi lo usa
development	la vostra macchina	`localhost:3001`	voi
preview	una PR	staging	revisori
staging	come produzione, dati del seed	staging	QA, cliente
production	clienti veri	produzione	tutti

- Staging ha il suo file `.db` popolato da `db:seed`: si può svuotare senza paura, ed è lì che si provano le migrazioni **prima** della produzione.
- **Feature flag**: funzionalità nascoste dietro un interruttore (Unleash, LaunchDarkly, o un campo nella risposta dell'API). Si rilascia il codice senza rilasciare la funzionalità.
- Versione visibile nell'interfaccia: `import.meta.env.VITE_APP_VERSION`, iniettata dalla CI dal tag git. Senza, nessuno sa quale build ha davanti.

Errori: Sentry (o simili)

```
npm install @sentry/react -w frontend
```

```
import * as Sentry from '@sentry/react';
Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  release: `minicommerce-web@${import.meta.env.VITE_APP_VERSION}`,
  environment: import.meta.env.MODE,
  integrations: [Sentry.browserTracingIntegration(), Sentry.replayIntegration()],
  tracesSampleRate: 0.1, replaysOnErrorSampleRate: 1.0,
});
// nell'Error Boundary del modulo 1: onError={(e) => Sentry.captureException(e)}
```

- Con le **source map** caricate in CI (e non pubblicate accanto al bundle), lo stack trace mostra il vostro TypeScript, non il codice minificato.
- Oggi un errore si vede solo nel toast dell'utente: nessuno, lato vostro, sa che è accaduto.
- Vale anche per il backend (`@sentry/node`): meglio di `console.error` in un log che nessuno legge.

Prestazioni reali, analytics, uptime

```
import { onLCP, onINP, onCLS, type Metric } from 'web-vitals';
// POST /api/vitals non esiste ancora nel backend: aggiungerlo e' parte del laboratorio
const invia = (m: Metric) => navigator.sendBeacon('/api/vitals',
  JSON.stringify({ name: m.name, value: m.value, page: location.pathname }));
onLCP(invia); onINP(invia); onCLS(invia);
```

- Le metriche di laboratorio (Lighthouse) non bastano: i dati reali mostrano i telefoni lenti e le reti pessime. Il catalogo con le immagini base64 si misura **lì**, non sul vostro portatile.
- Analytics rispettosi della privacy (**Plausible**, **Umami**, Fathom): niente cookie, niente banner.
- Uptime: un ping ogni minuto su `GET /api/health` e sul sito, con avviso su Telegram o Slack. L'endpoint esiste già: usatelo.

Versioni e changelog

```
npm install -D @changesets/cli && npx changeset init   # nella root del monorepo
npx changeset              # "cosa e' cambiato?" -> un file in .changeset/
npx changeset version      # aggiorna le versioni e i CHANGELOG.md
git tag v1.3.0 && git push --tags     # -> parte la release desktop
```

- **Semantic versioning**: MAJOR rompe, MINOR aggiunge, PATCH corregge. L'auto-update desktop e mobile si basa su questo.
- In un monorepo i changeset dicono *quali* pacchetti cambiano: toccate `shared` e vi ricordano che backend, frontend, desktop e mobile ne dipendono.
- Il changelog lo leggono gli utenti (note di rilascio) e i colleghi (cosa è successo fra due tag).
- Convenzione dei commit (`feat:`, `fix:`) $\rightarrow$ changelog generabile automaticamente.

Laboratorio 7

Laboratorio: Dal commit all'utente senza toccare nulla a mano

1. Monorepo con npm workspaces + Turborepo: `backend`, `frontend`, `packages/shared` con i tipi del contratto (`Product`, `Category`, `AuthUser`, `Cart`, `CartItem`, `MockOrder`).
2. Aggiungere gli script mancanti (`typecheck`, `lint` al backend) e una CI che su ogni PR esegue lint, tipi, test, build e `npm audit`.
3. Deploy automatico su push in `main`: `frontend/dist` su hosting statico, backend in container con volume persistente e segreti dall'ambiente.
4. Una release desktop su tag con `tauri-action` (almeno Windows e Linux).
5. Sentry con source map; `web-vitals` inviate a `POST /api/vitals`, da aggiungere.

Domande di verifica

Perché `npm ci` e non `npm install` in CI? Cosa cacha Turborepo? Perché il backend non può girare in due istanze? Cosa accade se `NODE_ENV` non è `production`?